49

BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

15. İlke: Tüm İşler Bir Use Case Değişikliğidir (Özellik, Değişiklik, Hata)

Principle 15: All Work is a Change to a Use Case (Features, Changes, and Bugs)

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              15. İLKE: HER İŞ BİR USE CASE DEĞİŞİKLİĞİDİR              │
└──────────────────────────────────────────────────────────┘

Masaüstünüze Gelen İş Türleri:

1. YENİ ÖZELLİK (New Feature) ──► Sisteme YENİ BİR USE CASE ekler!
   - Gereksinim kataloğunda (requirements.md) yeni bir kullanıcı hikayesi açar.

2. DEĞİŞİKLİK TALEBİ (Change)  ──► MEVCUT BİR USE CASE'İ günceller!
   - Bir iş kuralı (BR) değişir, yeni bir alternatif akış (A4) eklenir.

3. YAZILIM HATASI (Bug Report) ──► KOD İLE ŞARTNAME ARASINDAKİ ÇELİŞKİDİR!
   - Ya kod şartnameden sapmıştır (kod düzeltilir).
   - Ya da şartname eksik kalmıştır (şartname düzeltilir ve kod yenilenir).

🎯 MİMARİ İLKE:
"Hiçbir kod değişikliği, şartnameden geçmeden sisteme GİREMEZ!
 Aksi halde şartname çürümeye başlar ve kod tekrar tek gerçek haline gelir!"
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphaneyi düşünün: Kütüphaneye yeni bir kitaplık eklemek (Yeni Özellik), mevcut kitaplığın sırasını değiştirmek (Değişiklik) veya yanlış rafa konmuş bir kitabı doğru rafa koymak (Hata)... Hepsi kütüphane kataloğuna (Use Case Şartnamesi) işlenmek zorundadır! Eğer rafa yeni kitap koyup kataloğa yazmazsanız, o kütüphane 1 hafta sonra çöplüğe döner!

🇬🇧 ENGLISH (ELI5)

Think of a grand library: Adding a new science section (New Feature), rearranging the history shelves (Change Request), or reshelving a misplaced book (Bug fix)... All three require updating the master library catalog (Use Case Spec)! If you move books without updating the catalog, the library descends into chaos within a week.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

15. İlke: Tüm İşler Bir Use Case Değişikliğidir (All Work is a Change to a Use Case): Yazılım projelerinde işler geliştiricilerin masasına tertemiz sıfır kilometre use case'ler halinde gelmez; Pazartesi bir değişiklik talebi, Salı günü ise bir bug raporu olarak düşer. AI Unified Process'te her üç iş türü de temelde aynı şeydir: Bir use case değişikliği! 1) Yeni Özellik: Sisteme yeni bir use case (UC-XXX) ekler veya mevcut olanı yeni aktör ve akışlarla genişletir. 2) Değişiklik Talebi: Var olan use case'i revize eder; örneğin 28 günlük ödünç süresini 14 güne düşürür (BR-004 güncellemesi). 3) Bug: Şartname ile kod arasındaki uyuşmazlıktır. Çoğunlukla kod yanlıştır; bazen de şartname paydaşın gerçek iradesini eksik yansıtmıştır. Çözüm her halükarda önce şartnamenin incelenmesi ve güncellenmesiyle başlar. Bu disiplin korunmazsa 'Principle 11' (Şartname koddan daha uzun yaşar) anında çöker.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 15: All Work is a Change to a Use Case: Software engineering reality rarely delivers pristine greenfield specifications. Work arrives as frantic Jira tickets, customer change requests, or production incidents. In the AI Unified Process, all three are structurally identical: a change to a use case specification. 1) New Features add use case specifications or attach new actors/flows. 2) Change Requests modify existing preconditions, business rules, or alternative flows. 3) Bugs represent disagreements between specification and reality. If a side-channel exists where developers patch production code directly without updating specifications, the specification rots and code reverts to being the fragile truth. By enforcing that every change flows specification-first, the architecture retains continuous regenerability.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Tüm İş Değişikliklerinin Şartname Tabanlı Mühürlenmesi
package com.maang.aiup.allwork;

public class WorkDeltaArchitecture {
    // İş Kuralı BR-004 Değişikliği: 28 günden 14 güne indirildi
    public record LoanPolicy(int maxDaysAllowed) {
        // Eski sürüm: maxDaysAllowed = 28
        // Değişiklik sürümü (CR-402): maxDaysAllowed = 14
        public static LoanPolicy defaultPolicy() {
            return new LoanPolicy(14); // Şartname güncellendi, kod yenilendi
        }
    }

    public static class LoanService {
        private final LoanPolicy policy;

        public LoanService(LoanPolicy policy) {
            this.policy = policy;
        }

        public int calculateDueOffset() {
            return policy.maxDaysAllowed();
        }
    }

    void main() {
        var service = new LoanService(LoanPolicy.defaultPolicy());
        System.out.println("✅ Şartname Değişikliği (CR-402) Koda Yansıdı: Yeni Vade = " +
service.calculateDueOffset() + " gün.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Acil bir bug çıktığında doğrudan koda girip 'quick fix' yapmayın! Önce şartnamedeki kuralı netleştirin, sonra /implement komutunu çalıştırın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never apply an unrecorded hotfix directly to production source code. Rectify the specification first, then execute generation to maintain single-truth traceability.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Yeni özellik, değişiklik veya bug fark etmez: Hepsi bir use case değişikliğidir.
- Şartnameyi atlayan her kod yaması, şartnamenin çürümesine ve projenin çökmesine yol açar.
- Değişiklik önce şartnamede onaylanır; ardından kod ve testler tek komutla yenilenir.

🎯 Key Architectural Takeaways

- Features, change requests, and bugs are all structurally a change to a use case.
- Any patch that bypasses the spec accelerates documentation rot and destroys regenerability.
- Changes are approved in the specification first; code and tests are regenerated deterministically.